Software Requirements Specification (SRS)

CampusCart — College Second-Hand Marketplace

Project Documentation for Academic Submission

Project Name	CampusCart
Project Type	College Second-Hand Marketplace
Technology	React, Vite, Node.js, Express, MongoDB/Mongoose (planned/configurable)

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for CampusCart, a web application concept designed to help college students discover and exchange second-hand items. It is intended for project review, development, testing, and academic submission.

1.2 Scope

CampusCart provides a marketplace interface where students can browse and search second-hand items such as textbooks, calculators, electronics, laptops, and college supplies. The planned system includes user registration and login, product listings, product details, seller contact, wishlist, and administrative moderation. The current demonstration uses a local eight-item sample catalogue when the product API is unavailable or returns HTTP 503. These demo items are not persisted in a database.

1.3 Intended Audience

The intended audience includes the student developer, project guide, faculty evaluators, testers, and future developers.

1.4 Definitions and Acronyms

MERN: MongoDB, Express.js, React, and Node.js. API: Application Programming Interface. JWT: JSON Web Token. SRS: Software Requirements Specification. Demo data: sample content used to demonstrate the interface without permanent database storage.

2. Overall Description

2.1 Product Perspective

CampusCart is a web-based client-server application. The frontend is built with React and Vite, with React Router for navigation and CSS for styling. The backend uses Node.js and Express. Mongoose is intended for MongoDB access. bcryptjs and JWT are intended for password hashing and authentication.

2.2 Product Functions

The intended system functions are: browse marketplace listings; search listings; filter listings by category and price; view product details; register and log in; create and manage product listings; contact sellers; save wishlist items; and provide administrative review or moderation.

2.3 User Classes

Visitor: can explore publicly available listings. Registered student: is intended to manage listings and use account-dependent features. Administrator: is intended to review or moderate listings and user-generated content.

2.4 Operating Environment

The application is designed to run locally on a computer using a modern web browser. Development requires Node.js and npm. The frontend runs through Vite, and the backend runs through Express. A configured MongoDB database is required for database-backed features.

2.5 Constraints and Assumptions

The project is a college-level prototype. The current local demonstration has been reported to show eight sample products, with working search, category/price filters, detail navigation, and loaded images. MongoDB is not configured, so persistent storage and authentication have not been verified. Availability of internet access may be required for remotely hosted images or a future hosted deployment.

3. Functional Requirements

FR-01: Browse Products

The system shall display marketplace products to visitors. In demo mode, it shall display a local sample catalogue when the product API is offline or returns HTTP 503.

FR-02: Search Products

The system shall allow users to search the available catalogue using a search field.

FR-03: Filter Products

The system shall allow users to narrow displayed products by category and price, where those controls are available.

FR-04: View Product Details

The system shall allow users to open a product details page from a listing.

FR-05: Demo Data Disclosure

The system shall clearly identify demonstration products as sample data and indicate that they are not permanently stored.

FR-06: Account Registration and Login

The planned system shall allow users to create accounts and log in securely. This requirement is not verified in the current demo because the database and JWT secret are not configured.

FR-07: Manage Listings

The planned system shall allow authenticated users to create, update, and remove their own product listings when backend persistence is available.

FR-08: Seller Contact

The planned system shall provide a way for interested users to contact a seller, subject to the implementation and privacy controls.

FR-09: Wishlist

The planned system shall allow authenticated users to save and revisit items, subject to database configuration.

FR-10: Administration

The planned system shall provide administrator controls for reviewing or moderating listings, subject to implementation and authorization.

4. Non-Functional Requirements

NFR-01 Usability

The interface should be straightforward for college students to understand and use.

NFR-02 Performance

The local demonstration should load the homepage and render the sample catalogue without unnecessary delay on a typical student computer.

NFR-03 Security

Passwords should be hashed with bcryptjs. Protected backend routes should use JWT-based authentication. Secrets and database credentials must be kept in the backend environment file and must not be committed to a public repository.

NFR-04 Reliability

The frontend should handle product API unavailability gracefully by using the configured demo-data fallback.

NFR-05 Maintainability

Frontend, backend, models, middleware, and routes should remain organized into separate project folders.

NFR-06 Compatibility

The website should work in a modern browser such as Chrome or Edge.

NFR-07 Data Integrity

When database integration is enabled, product and account data should be validated before being stored.

5. External Interface Requirements

5.1 User Interface

The website includes a landing page, marketplace browsing interface, search and filter controls, product cards, product detail pages, and account-related navigation.

5.2 Software Interfaces

The React frontend communicates with the Express backend through HTTP API requests. The backend is designed to use Mongoose to communicate with MongoDB. JWT and bcryptjs are intended for authentication and password security.

5.3 Hardware Interfaces

No special hardware is required beyond a computer or mobile device capable of running a modern web browser.

5.4 Communications

Local development uses HTTP on localhost. A production deployment should use HTTPS.

6. Data Requirements

6.1 Product Data

A product record is expected to contain fields such as product identifier, title, description, category, price, image, seller information, and listing status, depending on the implemented schema.

6.2 User Data

A user record is expected to contain an identifier, name, email, hashed password, and role or account status, depending on the implemented schema.

6.3 Current Demo Data

The current marketplace fallback contains eight sample listings. They are used for demonstration and are not persisted in MongoDB.

7. Use Cases

UC-01 Browse Marketplace

A visitor opens CampusCart, views available listings, and browses product cards.

UC-02 Find an Item

A visitor enters a search term or applies category/price filters to narrow the displayed listings.

UC-03 View Details

A visitor selects a listing and opens its detail page.

UC-04 Register or Log In

A student uses the account interface. Successful operation depends on backend configuration and is not verified in the current demo.

UC-05 Create a Listing

An authenticated student submits item information. This use case requires working authentication and database persistence.

8. Acceptance Criteria and Test Status

AC-01

The homepage opens in a browser. Reported as working in the current local environment.

AC-02

The marketplace shows eight demo listings when the API is unavailable or returns 503. Reported as verified by the development agent and should be rechecked before submission.

AC-03

Search and category/price filters operate on the demo catalogue. Reported as verified by the development agent and should be rechecked before submission.

AC-04

Selecting a demo listing opens its details page and identifies the data as not persisted. Reported as verified by the development agent.

AC-05

A production build completes successfully. Reported as passing by the development agent.

AC-06

MongoDB persistence, registration, login, wishlist persistence, and listing creation are not verified because MongoDB is not configured.

AC-07

No automated test suite is currently reported.

9. Limitations and Future Enhancements

9.1 Current Limitations

This submission version supports a local demonstration using sample catalogue data. A real MongoDB connection is not configured, and account authentication and persistent database operations have not been verified. Some planned features may therefore be unavailable in the current demonstration.

9.2 Future Enhancements

Configure MongoDB Atlas; configure a secure backend JWT secret; test registration and login; implement and test persistent listing management, seller contact, wishlist, and admin moderation; add automated tests; and deploy using HTTPS.

10. Conclusion

Conclusion

CampusCart demonstrates the user interface and core browsing experience of a college second-hand marketplace. The current prototype supports browsing sample listings, searching, filtering, and viewing product details. Database persistence and authentication remain future configuration and verification tasks.